

# STA 5635 HW 6

Palmer Swanson

February 19, 2026

## Problem 1

- (1a) Two random forest classifiers with 100 trees each. One with 22 features, the other with 500. Table in figure 1

| Model                       | Train Misclass. Error | Test Misclass. Error |
|-----------------------------|-----------------------|----------------------|
| Random Forest, 22 Features  | 0.000000              | 0.296667             |
| Random Forest, 500 Features | 0.000000              | 0.156667             |

Table 1: Train and Test Misclassification Errors, part a

- (1b) Compute feature importances using mean decrease in impurity (MDI) and the permutation feature importances.

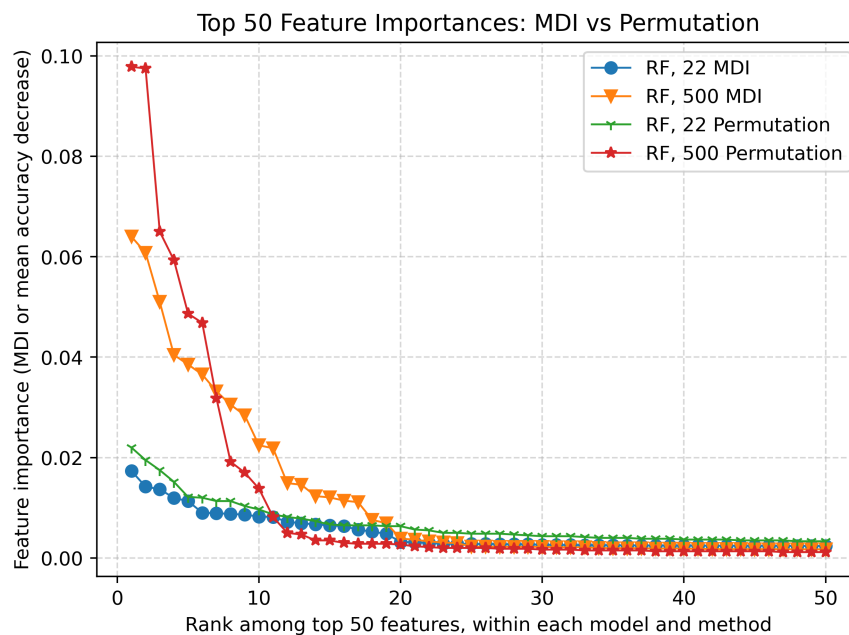


Figure 1: Feature Importances for the two RF, computed via MDI and via Permutations, part b.

- (1c) Selecting  $k \in \{10, 20, 30, 40\}$  most important features by the two methods mentioned above on the two classifiers from part a, train RF classifier where each split is from all 500 features. The test and train errors are reported in table 2.

| Label       | k  | Train Misclass. Error | Test Misclass. Error |
|-------------|----|-----------------------|----------------------|
| RF_22_MDI   | 10 | 0.000000              | 0.156667             |
| RF_22_MDI   | 20 | 0.000000              | 0.118333             |
| RF_22_MDI   | 30 | 0.000000              | 0.121667             |
| RF_22_MDI   | 40 | 0.000000              | 0.128333             |
| RF_500_MDI  | 10 | 0.000000              | 0.121667             |
| RF_500_MDI  | 20 | 0.000000              | 0.118333             |
| RF_500_MDI  | 30 | 0.000000              | 0.116667             |
| RF_500_MDI  | 40 | 0.000000              | 0.131667             |
| RF_22_PERM  | 10 | 0.000000              | 0.136667             |
| RF_22_PERM  | 20 | 0.000000              | 0.135000             |
| RF_22_PERM  | 30 | 0.000000              | 0.133333             |
| RF_22_PERM  | 40 | 0.000000              | 0.135000             |
| RF_500_PERM | 10 | 0.000000              | 0.120000             |
| RF_500_PERM | 20 | 0.000000              | 0.123333             |
| RF_500_PERM | 30 | 0.000000              | 0.130000             |
| RF_500_PERM | 40 | 0.000000              | 0.131667             |

Table 2: Training and test misclassification errors for the fitted models, part c.

- (1d) For  $k \in \{10, 20, 30, 40\}$ , two plots, one of each depth  $d \in \{4, 5\}$ .  $d = 4$  is figure 2,  $d = 5$  found in figure 3. Of the 320 classifiers, we can report the configuration with the lowest test error in table: 3

| Label       | k  | d | n_estimators | Train Misclass. Error | Test Misclass. Error |
|-------------|----|---|--------------|-----------------------|----------------------|
| RF_500_PERM | 10 | 5 | 80           | 0.031500              | 0.130000             |

Table 3: Configuration with smallest test error.

- (1e) Of the models, the random forest classifier from part c, which selects the top 30 features according to mean decrease in impurity (MDI) with all 500 features considered at each split, produced the lowest test misclassification error of 0.11667.

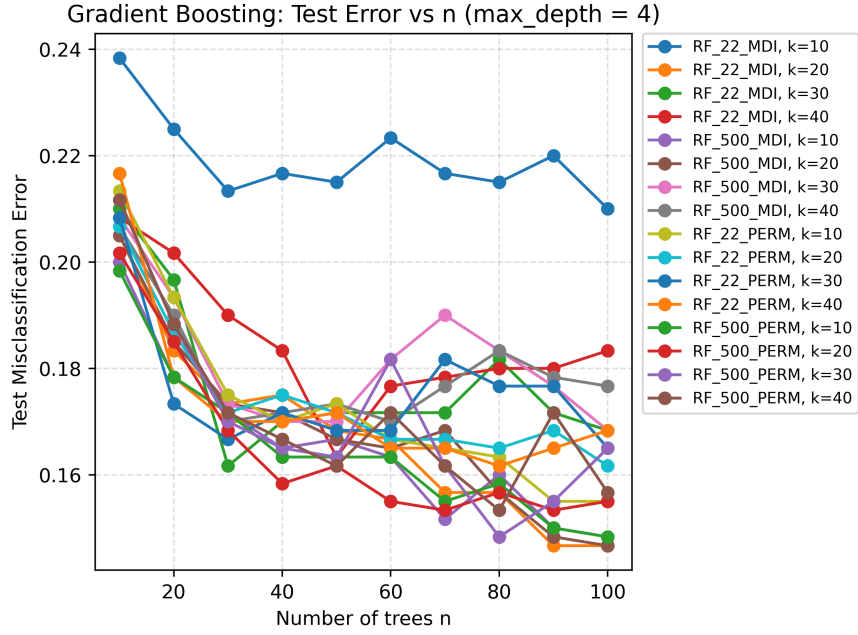


Figure 2: Test Errors vs.  $n$  for the 16 classifiers.  $d=4$

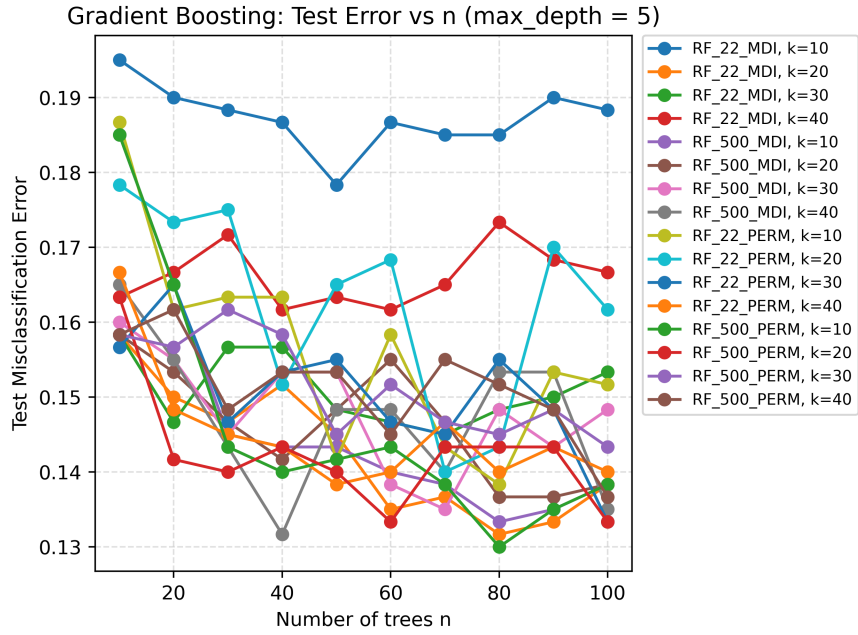


Figure 3: Test Errors vs.  $n$  for the 16 classifiers.  $d=5$

# Code

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.ensemble import RandomForestClassifier
4 from sklearn.ensemble import GradientBoostingClassifier
5 from sklearn.metrics import accuracy_score
6 from sklearn.inspection import permutation_importance
7 import matplotlib.pyplot as plt
8
9 #working with the Madelon Data
10 #loading in everything. Reusing the copy of the dataset from hw01
11 X_train = pd.read_csv("assignments/hw01/data/madelon_train.data",
12                       delim_whitespace=True, header=None)
13
14 X_valid = pd.read_csv("assignments/hw01/data/madelon_valid.data",
15                      delim_whitespace=True, header=None)
16
17 y_train = pd.read_csv("assignments/hw01/data/madelon_train.labels",
18                      header=None).values.ravel()
19 y_valid = pd.read_csv("assignments/hw01/data/madelon_valid.labels",
20                      header=None).values.ravel()
21
22 #part a. Need two random forests and need to fit them
23 rf_22 = RandomForestClassifier(n_estimators=100, max_features=22)
24 rf_500 = RandomForestClassifier(n_estimators=100, max_features=500)
25
26 #fitting
27 rf_22.fit(X_train, y_train)
28 rf_500.fit(X_train, y_train)
29
30 #making predictions
31 rf_22_pred_train = rf_22.predict(X_train)
32 rf_500_pred_train = rf_500.predict(X_train)
33
34 rf_22_pred_test = rf_22.predict(X_valid)
35 rf_500_pred_test = rf_500.predict(X_valid)
36
37 #errors
38 rf_22_train_error = 1.0 - accuracy_score(y_train, rf_22_pred_train)
39 rf_500_train_error = 1.0 - accuracy_score(y_train, rf_500_pred_train)
40
41 rf_22_test_error = 1.0 - accuracy_score(y_valid, rf_22_pred_test)
42 rf_500_test_error = 1.0 - accuracy_score(y_valid, rf_500_pred_test)
43
44 #need to put this in a table
45 part1 = pd.DataFrame({
46     "Model": [
```

```

42         "Random_Forest,_22_Features",
43         "Random_Forest,_500_Features"
44     ],
45     "Train_Misclass._Error": [
46         rf_22_train_error, rf_500_train_error
47     ],
48     "Test_Misclass._Error": [
49         rf_22_test_error, rf_500_test_error
50     ]
51 })
52
53 #need to send this table to LATEX
54 LATEX_table = part1.to_latex(
55     index=False,
56     caption=None,
57     label=None,
58     column_format="c" * part1.shape[1],
59     escape=False
60 )
61
62 #save latex table
63 with open("assignments/hw06/output/train_test_errors_a.tex", "w") as f:
64     f.write(LATEX_table)
65
66 #part b
67 #https://scikit-learn.org/stable/auto_examples/ensemble/plot_forest_importances.html
68 #https://stackoverflow.com/questions/26984414/efficiently-sorting-a-numpy-array-in-descending-
69 #mean decrease in impurity
70
71 MDI_22 = rf_22.feature_importances_
72 MDI_500 = rf_500.feature_importances_
73
74 #permutation stuff
75 PERM_22 = permutation_importance(rf_22, X_valid, y_valid, n_repeats=10, n_jobs=4)
76 PERM_500 = permutation_importance(rf_500, X_valid, y_valid, n_repeats=10,
77     n_jobs=4)
78
79 #need just the importances mean
80 PERM_22 = PERM_22.importances_mean
81 PERM_500 = PERM_500.importances_mean
82
83 #need to sort and take the first 50.
84 MDI_22_sorted = -np.sort(-MDI_22)[:50]
85 MDI_500_sorted = -np.sort(-MDI_500)[:50]
86 PERM_22_sorted = -np.sort(-PERM_22)[:50]
87 PERM_500_sorted = -np.sort(-PERM_500)[:50]

```

```

88 ranks = np.arange(1, 51)
89
90 plt.figure()
91 plt.plot(ranks, MDI_22_sorted, marker="o", linewidth=1, label="RF, 22 MDI")
92 plt.plot(ranks, MDI_500_sorted, marker="v", linewidth=1, label="RF, 500 MDI")
93 plt.plot(ranks, PERM_22_sorted, marker="1", linewidth=1, label="RF, 22
    Permutation")
94 plt.plot(ranks, PERM_500_sorted, marker="*", linewidth=1, label="RF, 500
    Permutation")
95
96 #Axis is a little confusing bc we're overlaying various models (RFs of different
    feature sizes)
97 #and two different methods of determining feature importance
98 plt.xlabel("Rank among top 50 features, within each model and method")
99 plt.ylabel("Feature importance (MDI or mean accuracy decrease)")
100 plt.title("Top 50 Feature Importances: MDI vs Permutation")
101 plt.grid(True, linestyle="--", alpha=0.5)
102 plt.legend()
103 plt.tight_layout()
104 plt.savefig("assignments/hw06/figures/feature_importances.png", dpi=400,
    bbox_inches="tight")
105 plt.close()
106 #plt.show()
107
108 #part c
109 #ok 16 classifiers. 2 RFs (22 and 500), 2 methods for determining importance
110  #(DMI or Permutation), and 4 k values. Sounds like a loop is needed.
111
112 #store the four importance items from the four combinations of RFs and Methods
113 #maybe put these in a dict or something to make sure that the names match the
114 #method
115 methods = [MDI_22, MDI_500, PERM_22, PERM_500]
116 names = ["RF_22_MDI", "RF_500_MDI", "RF_22_PERM", "RF_500_PERM"]
117 k_values = [10, 20, 30, 40]
118
119 rows = []
120
121 for i in range(4):
122     #get the method
123     import_class = methods[i]
124     identifier = names[i]
125     for k in k_values:
126         #need to get the top k features AND their indices
127         #https://numpy.org/doc/2.1/reference/generated/numpy.argsort.html
128         k_selected = np.argsort(-import_class)[:k]
129
130         #get the columns of the data that match these indices

```

```

131     X_trains = X_train.iloc[:, k_selected]
132     X_valids = X_valid.iloc[:, k_selected]
133
134     #can still use the y data
135     #train the RF using those k features
136     k_rand_forest = RandomForestClassifier(n_estimators=100,max_features=k)
137
138     k_rand_forest.fit(X_trains, y_train)
139
140     #predictions
141     train_predictions = k_rand_forest.predict(X_trains)
142     test_predictions = k_rand_forest.predict(X_valids)
143
144     #misclass error
145     train_error = 1.0 - accuracy_score(y_train, train_predictions)
146     test_error = 1.0 - accuracy_score(y_valid, test_predictions)
147
148     #add these to the rows
149     rows.append({
150         "Label": identifier,
151         "k": k,
152         "Train_Misclass_Error": train_error,
153         "Test_Misclass_Error": test_error
154     })
155
156 part3 = pd.DataFrame(rows)
157
158 #smallest test error is going to be here, find it
159 smallest_test_error = part3[part3['Test_Misclass_Error'] == part3['Test_
    Misclass_Error']].min()
160 print(smallest_test_error)
161
162 #need to send this table to LATEX
163 LATEX_table = part3.to_latex(
164     index=False,
165     caption=None,
166     label=None,
167     column_format="c" * part3.shape[1],
168     escape=True
169 )
170
171 #save latex table
172 with open("assignments/hw06/output/train_test_errors_c.tex", "w") as f:
173     f.write(LATEX_table)
174
175 #part d. Big one. Just add another loop inside part c?
176 depths = [4, 5]

```

```

177 n_s = list(range(10, 110,10))
178
179 results = []
180
181 #largely the same as part c
182 for i in range(4):
183     #get the method
184     import_class = methods[i]
185     identifier = names[i]
186     for k in k_values:
187         #need to get the top k features AND their indices
188         #https://numpy.org/doc/2.1/reference/generated/numpy.argsort.html
189         k_selected = np.argsort(-import_class)[:k]
190
191         #get the columns of the data that match these indices
192         X_trains = X_train.iloc[:, k_selected]
193         X_valids = X_valid.iloc[:, k_selected]
194
195         for d in depths:
196             for n in n_s:
197                 #creating Booster with n decision trees and d maximum depths
198                 clf = GradientBoostingClassifier(n_estimators=n, max_depth=d)
199
200                 clf.fit(X_trains, y_train)
201
202                 #make predictions
203                 train_predictions = clf.predict(X_trains)
204                 test_predictions = clf.predict(X_valids)
205
206                 #errors
207                 train_error = 1.0 - accuracy_score(y_train, train_predictions)
208                 test_error = 1.0 - accuracy_score(y_valid, test_predictions)
209
210                 #add this to the results
211                 results.append({
212                     "Label": identifier,
213                     "k": k,
214                     "d": d,
215                     "n_estimators": n,
216                     "Train_Misclass_Error": train_error,
217                     "Test_Misclass_Error": test_error
218                 })
219
220 results_d = pd.DataFrame(results)
221
222 #plotting stuff. 16 curves on the same plot, for each d. So two plots
223 for d in depths:

```



```

224 plt.figure()
225 temp = results_d[results_d["d"] == d]
226
227 #each label is a curve. Need to work on this loop portion
228 for label in temp["Label"].unique():
229     for k in temp["k"].unique():
230         #extract the one that we want
231         holder = temp[(temp["Label"] == label) & (temp["k"] == k)]
232
233         #plot the single curve and give it the appropriate label
234         plt.plot(
235             holder["n_estimators"].values,
236             holder["Test_Misclass_Error"].values,
237             marker="o",
238             label=f"{label}, k={k}"
239         )
240
241     plt.xlabel("Number_of_trees_n")
242     plt.ylabel("Test_Misclassification_Error")
243     plt.title(f"Gradient_Boosting:_Test_Error_vs_n_(max_depth={d})")
244     plt.grid(True, linestyle="--", alpha=0.4)
245     #legend was too crowded
246     plt.legend(
247         fontsize=8,
248         ncol=1,
249         bbox_to_anchor=(1.02, 1),
250         loc="upper_left",
251         borderaxespad=0
252     )
253     plt.tight_layout()
254     #plt.show()
255     plt.savefig(f"assignments/hw06/figures/gb_test_error_depth_{d}.png",
256                 dpi=400, bbox_inches="tight")
257     plt.close()
258
259 #need to find row containing the smallest test error. Returns all rows, so if
260 there are ties,
261 #we'd see them
262 smallest_test_error = results_d[results_d['Test_Misclass_Error'] ==
263                                 results_d['Test_Misclass_Error'].min()]
264
265 #save this to LATEX
266 #need to send this table to LATEX
267 LATEX_table = smallest_test_error.to_latex(
268     index=False,
269     caption=None,
270     label=None,

```

```
268     column_format="c" * smallest_test_error.shape[1],
269     escape=True
270 )
271
272 #save latex table
273 with open("assignments/hw06/output/smallest_test_error.tex", "w") as f:
274     f.write(LATEX_table)
```